



Index Table

S.no.	Topic name
1.	Introduction to Table
2.	Out of the Box Tables
3.	Types of Tables in ServiceNow
4.	Extended and Referenced Table
5.	Custom Tables
6.	Schema Map
7.	Dictionary Entries
8.	Configure label and choice
9.	Configure styles
10.	Choice list
11.	Data dictionary tables
12.	Use of Task table
13.	Dictionary Override

Tables & Relationships Tables Creation

1. Introduction to Table

A **Table** is a collection of records (rows) and fields (columns) that stores data. It is the core building block of the ServiceNow database.

- **Records (Rows):** Each entry in a table is a record. Every record is identified by a unique 32-character GUID called **sys_id**.
- **Fields (Columns):** Data elements within a record (e.g., Name, Status, Date).
- **Backend Name vs. Label:**
 - **Label:** User-friendly name visible on the UI (e.g., "Incident").
 - **Name:** Database name used by the system (e.g., incident).
- **Naming Convention (Crucial):**
 - **System Tables:** No prefix (e.g., incident, user).
 - **Global Custom Tables:** Prefixed with **u_** (e.g., u_car_requests).
 - **Scoped Custom Tables:** Prefixed with Scope ID (e.g., x_acme_app_table).
- **Key Backend Table: sys_db_object** (This table contains a record for every table in the system).

[**Warning:** Never change the *Name* of a table after creation if possible. It breaks scripts and references. You can safely change the *Label*.]

Use Case:

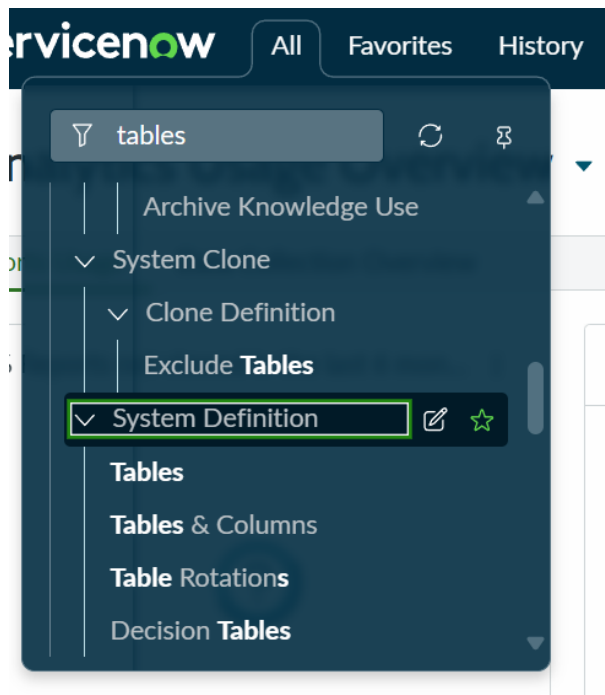
Analogy: Think of a Table as a specific tab in an **Excel Workbook**.

- The tab name is the **Table Name**.
- Column A, B, C are the **Fields**.
- Rows 1, 2, 3 are the **Records**.
- The system needs to know if the tab is standard (provided by Excel) or created by you (Custom).

Example:

Creating a new custom table for tracking "Office Visitors".

- **Label:** Office Visitors
- **Name:** u_office_visitors (Auto-generated)
- **Extends:** None



2. Out-of-the-Box (OOB) Tables

OOB Tables are tables that come pre-installed with the ServiceNow platform. They contain built-in business logic, fields, and workflows.

- **Task (task):** The parent table for all work-related processes.
- **Configuration Item (cmdb_ci):** The base table for the CMDB.
- **User (sys_user):** Stores employee/user details.
- **Group (sys_user_group):** Stores teams and queues.
- **Location (cmn_location):** Stores physical addresses/sites.

Admin Impact: Always check if an OOB table exists before creating a custom one. Using OOB tables ensures you get future updates from ServiceNow and compatibility with standard features.

{ **CMDB table** is part of a larger database that stores information about an organization's IT assets, services, and their relationships }

Use Case:

Analogy: Pre-fabricated Housing Modules.

Instead of building a kitchen from scratch (laying pipes, wiring), you buy a pre-made kitchen module (OOB Table) that already has water, gas, and electricity hookups. You just paint the walls (Configure it).

3. Types of Tables in ServiceNow

Tables are categorized based on their inheritance and origin.

Table Type	Definition	Example
Base Table	A table that does not extend any other table. It is the root of a hierarchy.	task, cmdb, sys_user
Core Table	Standard tables provided by ServiceNow (can be base or extended).	incident (extends task), problem (extends task)
Custom Table	Created by Admins/Developers.	Any custom table like visitors, spending, etc.
Many-to-Many (M2M)	A special table that links two other tables together, allowing multiple records from Table A to relate to multiple records in Table B.	sys_user_grmember (Links Users to Groups)
Database View	Not a physical table, but a virtual joined view of two tables for reporting.	incident_metric

Use Case

Analogy: Organization Chart.

- **Base Table:** The CEO (Reports to no one).
- **Core Table:** VP of Sales (Standard role, reports to CEO).
- **Custom Table:** "Head of AI Research" (New role created specifically for this company).

The screenshot shows the ServiceNow 'Tables' list view. A red box highlights the 'Task' table. Red arrows point from the 'Task' table to the 'Base Configuration Item' table with the annotation 'all are base tables'. Another red arrow points from the 'Task' table to the 'Interaction' table with the annotation 'Not extending any table'. A third red arrow points from the 'Task' table to the 'Automation Error Message' table with the annotation 'but extensible for other tables'.

	Label	Name	Extends table	Extensible	Updated
☐	Base Configuration Item	cmdb	(empty)	true	2018-09-13 01:57:50
☐	Interaction	Interaction	(empty)	true	2018-09-05 11:45:22
☐	MID Server Extension Context	err_agent_ext_context	(empty)	true	2018-09-13 02:07:20
☐	Log Entry	sys_log	(empty)	true	2018-09-13 02:10:12
☐	Automation Error Message	automation_error_msg	(empty)	true	2019-03-07 09:41:29
☐	Flow Connector	sa_flow_connector	(empty)	true	2019-03-07 09:42:16
☐	Task	task	(empty)	true	2018-09-13 02:18:09
☐	Create Field	v_field_creator	(empty)	true	2018-09-13 02:06:16
☐	Network Infrastructure Item	discovery_net_base	(empty)	true	2019-03-07 09:41:47

4. Extended Table and Referenced Tables

1. Extended Tables (Parent-Child):

- **Concept:** The Child table inherits **all** fields, business rules, and policies from the Parent.
- **sys_class_name:** This system field tells the database which table classification the record belongs to.
- **Admin Note:** If you add a field to the Parent (Task), it appears on **all** Children (Incident, Problem, Change). Use with caution!

2. Referenced Tables (Pointer):

- **Concept:** A field in Table A stores the sys_id of a record in Table B. No fields are inherited.
- **Example:** The "Caller" field on Incident references the sys_user table.

- **Dictionary Type:** Reference.

The screenshot shows a ServiceNow form with a 'Caller' field set to 'Carol Coughlin'. A red arrow points from this field to a 'User' modal window. The modal contains the following fields:

User	
User ID	carol.coughlin
First name	Carol
Last name	Coughlin
Title	
Department	Sales
Password	
Password needs reset	☐
Locked out	☐
Active	☒
Web service access only	☐
Internal Integration User	☐
Email	carol.coughlin@example.com
Notification	Disable
Calendar integration	
Time zone	
Date format	
Business phone	
Mobile phone	
Photo	

(Here in this depiction ' Caller ' field is pointing to the User (sys_user) table)

Use Case:

- **Extending: Biology/Genetics.** You (Child) extend your Parents. You inherit their DNA (Fields) automatically. You are a specific "Class" of your family.
- **Referencing: Library Card.** Your library card (Incident) *references* you (User). The card points to you, but the card is not *biologically related* to you.

Example:

- **Extending:** Incident extends Task.
- **Referencing:** Incident "Assignment Group" references sys_user_group.

5. Custom tables

Custom tables in ServiceNow are user-created tables that store data specific to an organization's unique needs, going beyond the platform's pre-built, or "out-of-the-box" tables. They allow for fully customized fields, workflows, and relationships, and are identified by an internal name starting with "u_". Custom tables are essential for managing data that doesn't fit within existing ServiceNow structures and require a license entitlement for use on a production instance.

5.1) Creating a Custom Table

Creating a custom table allows you to store and manage data specific to your organization's needs that don't fit into the existing core tables. These are created inside an Application in ServiceNow. When the "Add module" option is checked during table creation, ServiceNow automatically creates an Application Menu and a Module for the table – this completes the basic "application designing" from an Administrator perspective.

Step-by-Step Procedure

1. Navigate to **All** → **System Definition** → **Tables**.
2. Click the **New** button.
3. Fill out the required fields (detailed below).
4. Click **Submit** to create the table and its default fields.
5. After creation, the system redirects you to the **Table** form where you can further configure **Columns** (fields), **Access Controls**, and **Application Access**.

The screenshot shows the ServiceNow 'Table - New Record' form. The form is divided into two main sections: 'Table' and 'Columns'. The 'Table' section contains fields for Label, Name, Extends table, Application, Remote Table, Create module, Create mobile module, Add module to menu, and New menu name. The 'Columns' section is currently active, showing a list of columns with a search bar and a 'Table Columns' button. The form is titled 'Table - New Record' and has a 'Submit' button.

Table Section:

- 1 * Label: [Text Input]
- 2 * Name: [Text Input]
- 3 Extends table: [Text Input]
- 6 Application: Technova Project Management
- Remote Table: ☐
- Create module: ☒ 8
- Create mobile module: ☒
- 4 Add module to menu: -- Create new --
- New menu name: [Text Input]

Columns Section:

- 9 Columns: [Tab]
- 10 * Controls: [Tab]
- Application Access: [Tab]
- Table Columns: [Text Input]
- for text: [Text Input]
- Search: [Text Input]

Dictionary Entries

No templates are available. [Create A New One?](#)

Table New record View: Tables

Extends table

request desc

Create module ☒

Create mobile module ☒

Add module to menu -- Create new --

New menu name

Columns Controls Application Access

Extensible ☐

Live feed ☐

Use auto-numbering to define a sequential identifying code made up of a prefix, a base number and a padding value to ensure a consistent format

5 Auto-number ☐

Security Rules (ACLs) are required if anyone other than an administrator needs to work with this table. Creating default security rules will grant full access to this table to anyone with the user role you specify.

No templates are available. [Create A New One?](#)

Table Creation Form Fields/Settings

Field/Setting	Description	Admin Focus/Impact
1. Label	The user-friendly name displayed in the UI (e.g., "Employee Onboarding Request").	This will be the name of the new module, form title, and list title.
2. Name	The unique, backend name of the table, prefixed automatically (e.g., u_employee_onboarding_request).	Used for scripting, APIs, and backend searches. Must be unique across the instance.
3. Extends Table	Specifies an existing table to inherit fields and logic from (e.g., Task or Configuration Item).	Crucial administrative decision. Dictates initial set of fields, workflows, and security. (See Section 6)
4. Add module	(Checked by default) Creates a new Application Menu and a List Module for the	Makes the table and its data accessible via the

	new table.	Application Navigator for users.
5. Auto-number	(Checked by default) Creates an auto-incrementing number field (Number) for records, using a defined prefix (e.g., ENB).	Essential for easy record identification and referencing. Only available if not extending a table that already has a Number field (like Task).
6. Application	Specifies the application scope the table belongs to (e.g., Global or a custom Scoped Application).	Defines the table's prefix (u_ for Global, or app-specific for Scoped) and its security/data separation boundaries. (See Section 4)
7. User roles (Visible after saving table)	Specifies which roles will have read, write, and create access to the new table.	Security essential. Automatically creates basic ACLs for the table.
8. Create Module	Automatically creates a navigation module for the table so it becomes visible and accessible from the Application Navigator.	Enables users to access and manage table records directly from the Application Navigator, making the table visible and usable in the application UI.
9. Create access controls	(Checked by default) Automatically creates the basic CRUD (Create, Read, Update, Delete) Access Control rules based on the roles selected.	Best practice is to leave this checked. Ensures security is in place from the start. (See Section 7)

10. Application Access	Controls which applications/scopes can create/read/update/delete data of this table	Must set properly for cross-scope data usage

Application Access Options:

Accessible from - This application scope / All application scopes

CRUD rights for Create / Read / Update / Delete for cross Scope - Application Access.

Field Types List and Usage

Field Type	Data Category	Usage / Description	Practical Admin Example & Rationale
String	Text/Code	General-purpose short text input (default max length 40).	<i>Phone Number, User ID, Short Description.</i> Rationale: Most common type; use a small Max Length for performance.
String - Full UTF8	Text/Code	Stores text with complex or non-Latin character sets (e.g., globalized names, emojis).	<i>Product Name in multiple languages.</i> Rationale: Necessary for globalized instances where standard encoding may fail.
Integer	Numeric	Stores whole numbers (positive or	<i>Quantity of Assets, Floor Number.</i> Rationale: Used

		negative).	for strict counting or simple mathematical operations.
Decimal	Numeric	Stores numbers that include decimal places.	<i>Cost, Currency Rate, Percentage</i> . Rationale: Essential for accurate financial or non-integer metric calculations.
Price	Currency	Specialized field for currency values that supports multiple currency types and exchange rates.	<i>Hardware Purchase Price, Service Catalog Price</i> . Rationale: Necessary for organizations operating across different geographical markets.
True/False	Binary	A simple binary choice, presented as a checkbox. Stores true (1) or false (0).	<i>Active, Is Manager, Needs Approval</i> . Rationale: Most efficient way to store yes/no decisions.
Choice	Selection	A dropdown list where users select a single, pre-defined value.	<i>Incident Category, Priority, State</i> . Rationale: Enforces data integrity and consistency for reporting and workflow. Supports Dependent Choices .
Reference	Relationship	Creates a pointer/link to a record on a different table, storing the target record's	<i>Assigned To</i> (references <code>sys_user</code>), <i>Configuration Item</i> (references <code>cmdb_ci</code>). Rationale: Facilitates relationships

		sys_id.	without data duplication.
List	Relationship	Stores multiple references (a comma-separated list of sys_ids) to records on the same target table.	<i>Watch List, Related Incidents.</i> Rationale: Allows a single field to hold multiple pointers, creating a Many-to-Many appearance on a form.
Date	Time/Chrono	Stores only the date (YYYY-MM-DD). Does not include time.	<i>Birth Date, Contract Expiration Date.</i> Rationale: Used for non-time-sensitive chronological data.
Date/Time	Time/Chrono	Stores both the date and time (YYYY-MM-DD HH:MM:SS).	<i>Created On, Scheduled Start, Resolved At.</i> Rationale: Critical for calculating SLAs and precise reporting metrics.
Duration	Time/Chrono	Stores an elapsed period of time (e.g., 3 days, 4 hours).	<i>Time Worked, SLA Pause Duration.</i> Rationale: Used in calculations to represent a length of time.
Journal	Large Text/Log	Stores a timestamped, append-only chronological log of text entries.	<i>Work Notes, Additional Comments.</i> Rationale: Used for historical audit trails of communication; entries cannot be edited once saved.
HTML	Formatted	Stores text with rich	<i>Knowledge Article Body,</i>

	Text	formatting (bold, italics, lists) using an embedded WYSIWYG editor.	<i>Email Notification Body</i> . Rationale: Used where text requires visual structure and presentation.
Script	System/Code	Allows the administrator to enter and store JavaScript code directly in the record.	<i>Business Rule Condition Field, Transform Map Script</i> . Rationale: Used to embed executable logic within configuration records.

When a table is created, the system automatically creates several fields to manage record metadata, known as **System Fields**.

Full System Fields (Always Created)

These fields provide critical metadata about every record in the table.

Field Name	Description
sys_id	The unique 32-character identifier for every record in ServiceNow. This is the primary key.
sys_created_on	The date and time the record was first created.
sys_created_by	The User ID of the user who created the record.

sys_updated_on	The date and time the record was last modified.
sys_updated_by	The User ID of the user who last modified the record.
sys_class_name	The table that the record belongs to (used for table inheritance).
sys_mod_count	A counter that increments every time the record is saved/updated.
sys_tags	Allows users to add personal tags to records for quick filtering.

(These two fields appear only if Domain Separation is enabled)

sys_domain	Specifies the domain to which the record belongs
sys_domain_path	Defines hierarchical domain structure for access scope

Extra Fields When Extending Task

If your custom table extends the **Task [task]** table, it inherits all fields from **Task** and its parent tables.

Field Name	Description	Admin Focus
Number	The unique, auto-generated task number (e.g., INC1234567).	Provides the standard identifier for records.
Short	A brief summary of the task.	Standard field used for

Description		record Display .
State	The current lifecycle status of the task (e.g., New, In Progress, Closed).	Critical for task workflow and reporting.
Assignment Group	The group responsible for resolving the task.	Facilitates routing and workload management.
Assigned To	The individual user responsible for the task.	Facilitates routing and workload management.
Active	Indicates if the task is currently open or resolved.	Used for list filtering and reporting.

5.3) Deleting Custom Tables

Deleting a table is a **permanent, high-impact action**.

Rules, Dependency Check, and Data Impact

Permanent Data Loss: Deleting a table **permanently deletes all records** stored in that table. This action is **irreversible**.

Dependency Check: ServiceNow runs a check to warn or block deletion if critical dependencies (e.g., other tables referencing it, Business Rules, Reports) are found.

Schema Impact: The table definition is removed from `sys_db_object` and field definitions from `sys_dictionary`.

Best Practice for Administrators

AVOID Deleting in Production.

Best Practice: Deactivate or Archive Instead:

1. **Deactivate:** Uncheck the **Active** box on the table record.

2. **Archive:** Use the **Archiving** module to preserve historical data while removing it from the active table.
3. **Remove UI Access:** Delete the associated **Modules** and **Application Menu**.

5.4) u_ Prefix Explanation

Why Global Scope uses u_

Purpose: The prefix **u_** (for "User" or "Custom") is automatically added to the **Name** of any new field or table created in the **Global** scope.

Significance: It serves as a visual indicator that the element is **custom-created** by the customer/administrator.

How Scoped Applications Use App-Specific Prefix

Scoped Applications use a different, **unique prefix** based on the application's unique scope name (e.g., **x_acme_proj_mgmt_**).

Benefit: This enforces data separation and prevents naming collisions.

6. Schema Map

The **Schema Map** is a graphical tool to visualize relationships between tables.

- **Access:** System Definition → Tables → Select Table → "Show Schema Map" link.
- **Blue Bars:** The table extends or is extended by another (Inheritance).
- **Red Bars:** The table references or is referenced by another (Reference fields).
- **Use:** Vital for impact analysis. Before deleting a table or field, check the Schema Map to see what depends on it.

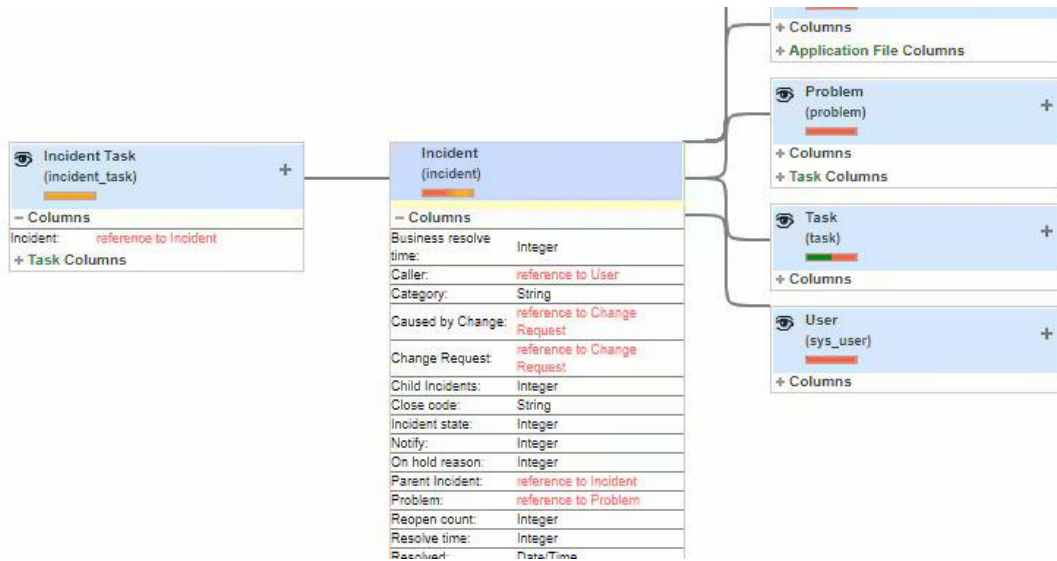
Use Case:

Analogy: Subway Map or Blueprints.

Instead of reading a list of station names (tables), you look at the map to see how lines connect, where they intersect, and where the "end of the line" is.

Example:

Viewing the Schema Map for the 'Incident' table to see its connection to Task and User.



7. Dictionary Entries

The **Dictionary** defines the details of every field (column) in every table.

- **Backend Table:** sys_dictionary.
- **What it controls:**
 - **Data Type:** String, Integer, Reference, Date/Time, Choice.
 - **Max Length:** Character limit (Default 40 for string).
 - **Mandatory:** Is data required?
 - **Read-Only:** Can users edit it?
 - **Display:** (True/False) Which field is shown when this record is referenced elsewhere? (e.g., User table uses "Name" as the Display field).
 - There can only be **one** Display=True field per table.

Use Case:

Analogy: Form Validation Rules.

When filling out a government form, the little text that says "Must be MM/DD/YYYY" or "Required Field" is the Dictionary Entry defining the rules for that box.

Dictionary Field Properties

Dictionary Entry
Caller View: Advanced

✎

✚

☰

⋮

Update

↑

↓

Dictionary entry manages how ServiceNow stores data in tables and fields (columns). For new dictionary entries, select a Table and the field Type of the new column. Also enter a column label, which becomes the field label, and the column name. If necessary, set a Max length for text String type fields, make the field Mandatory to save a record, and make the field a Display Value for reference fields so it appears on records that reference this table. [More Info](#)

* TableIncident [incident]

* TypeReference

* Column labelCaller

* Column namecaller_id

ApplicationGlobal

Active☒

Function field☐

Read only☐

Mandatory☐

Display☐

Controls the behavior of a field or functionality that depends on the field. [More Info](#)

Attributesencode_utf8=false,ref_contributions=user_show_incidents

Reference Specification

Choice List Specification

Dependent Field

Calculated Value

Default Value

Reference field specifies what table this field displays values from.

* ReferenceUser

Use reference qualifierSimple

Reference qual conditionAdd Filter ConditionAdd "OR" Clause

-- choose field -- -- oper -- -- value --

Reference Specification - Additional Customization

Property	Description	Admin Action/Impact
Display	If set to True for one field, that field's value is used to represent the record in reference lookups, breadcrumbs, and reports.	Only one field per table can have this set. Typically set on Short Description or Name.
Mandatory	Makes the field required; users cannot submit the form without providing a value.	Used to enforce data quality and completeness.
Read only	Prevents users from modifying the field's value, even if they have write access to the table.	Used for system-calculated values or static data.

Max length	For String fields, defines the maximum number of characters allowed.	Default is 40 . Crucial for performance; avoid setting an unnecessarily large length.
Dependent	Causes a field to be filtered based on the value of another field on the form.	Used for logical data filtering (e.g., filtering Subcategory choices based on Category).
Reference qualifier	A condition that filters the records shown in a Reference field's lookup list.	Basic Concept: A simple <i>Filter Condition</i> (e.g., Active = true) can be used to limit choices, improving usability and data integrity.

(**Note** - Only one field per table can be marked as Display = true.)

8. Configure Label and Choices

Admins often need to change how a field looks or what options it offers without changing the backend database structure.

- **Configure Label:** Changes the *Display Name* of the field for the users. Stored in **sys_documentation**.
 - *Note:* Does not change the backend column name.
- **Configure Choices:** Adds/Removes options in a dropdown menu. Stored in **sys_choice**.
 - **Personalize Choices:** Only affects the current user (avoid this).
 - **Configure Choices:** Affects **all** users (Admin action).

(**Note:** “Configure choices” option will only be visible on fields having type as “choice”.)

The screenshot shows a web application interface for managing projects. The form includes fields for Project Id (PROJECT0000116), Project Name, Project Status (dropdown), Project Size (dropdown), Expected Completion Date (calendar icon), Project Completed on (calendar icon), Project Manager (search icon), and Project Description. On the right, there are fields for 'No of developer needed', 'No of QA Engineer needed', and 'No of Architects needed', followed by an 'Impact' dropdown menu. A context menu is open over the 'Impact' dropdown, listing options: 'Configure Label', 'Configure Dictionary', 'Configure Styles', 'Show Security Rules', 'Configure Choices' (highlighted), 'Show Choice List', and 'Show - 'impact''. A 'Submit' button is at the bottom left. The footer indicates 'No templates are available' with a link 'Create A New One?'.

Use case with Analogy: Restaurant Menu.

- **Label:** Renaming "Patty Sandwich" to "Gourmet Burger" (The ingredients/backend didn't change, just the name).
- **Choices:** Adding "Vegan Cheese" to the list of available toppings.

Example:

Right-clicking a 'Category' field to add a new option called 'Hardware'.

9. Configure Styles

Field Styles allow Admins to change the color, font, or add visual dots to list views based on field values.

- **Backend Table:** sys_ui_style.

The screenshot shows the 'Projects' form in ServiceNow. The 'Impact' dropdown menu is open, displaying several configuration options. An arrow points to the 'Configure Choices' option.

Form fields visible:

- Project Id: PROJECT0000116
- Project Name: [Empty]
- Project Status: -- None --
- Project Size: -- None --
- Expected Completion Date: [Calendar icon]
- Project Completed on: [Calendar icon]
- Project Manager: [Search icon]
- Project Description: [Empty]
- No of developer needed: [Empty]
- No of QA Engineer needed: [Empty]
- No of Architects needed: [Empty]

Buttons: Submit

Footer: No templates are available. [Create A New One?](#)

- **Components:**

The screenshot shows the 'Style' form in ServiceNow. The 'Table' dropdown is set to '-- None --'. The 'Field name' and 'Value' fields are empty. The 'Style' and 'Themed style' fields are also empty.

Form fields visible:

- Table: -- None --
- Field name: [Empty]
- Value: [Empty]
- Style: [Empty]
- Themed style: [Empty]

Buttons: Submit

Footer: No templates are available. [Create A New One?](#)

- **Table:** Where to apply.
- **Field:** Which column to highlight.
- **Value:** The condition (e.g., javascript:current.priority==1).
- **Style:** CSS code (e.g., background-color:red).
- **Themed Style:** A themed style is a reusable set of style definitions that, when grouped with other styles in a theme, define the consistent look and feel of applications.
- **Common Use:** Showing a Red Dot or Red Background for "Critical" priority incidents.

Use Case:

Analogy: Traffic Lights / Highlighters.

Highlighting urgent emails in Red in your inbox so you notice them first.

Example:

Setting the 'Priority' field to display a red background when the value is '1 - Critical'.

10. Working with Choice List

Choice lists are specific field types where users select from pre-defined options.

- **Structure:**
 - **Backend Table name:** `sys_choice`
 - **Label:** What the user sees (e.g., "In Progress").
 - **Value:** What the database stores (e.g., 2 or in_progress).
 - **Sequence:** The order they appear in the dropdown.
- **Best Practice:** **NEVER** change the *Value* of an existing choice if data already exists. It will orphan historical data. Only change the *Label*.
- **Dependent Choices:** Making a choice list depends on another field (e.g., "Subcategory" options change based on "Category").

Use Case:

Analogy: State/Province Dropdown.

If you select "USA" in the Country field, the State dropdown only shows 50 US states. If you select "Canada", it shows Provinces. This is a Dependent Choice List.

Example:

* Select

* Choice List

Choice #1

Choice #2

Choice #3

Choice #4

* In-line Radio

☐ Choice #1 ☐ Choice #2 ☐ Choice #3 ☐ Choice #4

11. Data Dictionary Tables

To be an expert, you must know the backend tables that store the system's configuration.

Backend Name	Common Name	Purpose
sys_db_object	Tables	Defines the table itself (Name, Label, Extension).
sys_dictionary	Dictionary	Defines the columns/fields on the tables.
sys_documentation	Field Labels	Stores the human-readable labels for fields.
sys_choice	Choices	Stores the dropdown options for choice fields.

Admin Impact: If you are looking for a deleted field or debugging why a label isn't updating, you look directly in these tables.

Use Case:

Analogy: Library Catalog.

The books are the actual data (Incidents, Users). The Catalog (Data Dictionary) describes where the books are, what genre they are, and who wrote them. It is data about the data.

Example:

*Typing **sys_db_object.list** in the navigator to see the definition of every table in the system.*

12. Use of Task Table

The task table is the most important table in ServiceNow. It provides the core functionality for IT Service Management.

- **Why Extend Task?**
 - **You get free fields:** Number, Priority, State, Assignment Group, Assigned To, Short Description.
 - You get free features: **SLAs** (Service Level Agreements), **Approval Engines**, and **Activity Streams** (Journal fields) only work easily on tables extending Task.
- **When NOT to Extend Task:**
 - If the table is for storing data (like office locations, car models, software versions) that doesn't require "working" on it.

Use Case:

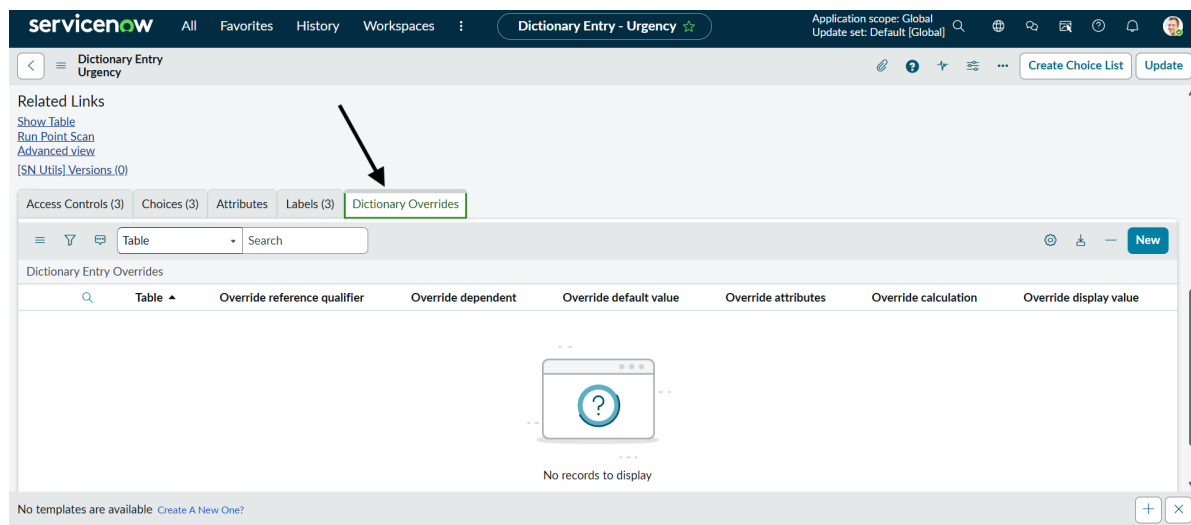
Analogy: Universal Job Application Form.

Whether you apply for Janitor, CEO, or Driver, the HR department uses a standard "Job Application" cover sheet (Task) that has Name, Date, and Desired Salary. Specific roles attach extra pages (Extended fields), but the core tracking is the same.

13. Dictionary Override

When a table extends Task, it inherits the dictionary settings of the parent. **Dictionary Override** allows the child table to change those settings *without* affecting the parent or other children.

- **Backend Table:** sys_dictionary_override.
- **What you can Override:**
 - **Default Value:** (e.g., Incident State defaults to "New", Change defaults to "Draft").
 - **Mandatory:** (e.g., "Configuration Item" is mandatory on Incident, but optional on Task).
 - **Read-Only status.**
 - **Reference Qualifier:** (Filtering lookup options).
- **What you CANNOT Override:** The **Data Type** (You can't change a String to an Integer).
- If you scroll down a little bit in a dictionary entry of any field, you will find the dictionary override.
- It is similar to the “ *Method Overriding*” in the OOP's concepts.



Use Case:

Analogy: House Rules.

- The "Parent" rule in the house is "Bedtime is 9 PM".
- The "Teenager" room (Child Table) has a Override: "Bedtime is 11 PM".

This specific rule only applies to the Teenager room; the rest of the house still follows the Parent rule.

Example:

Setting the 'Assignment Group' to be Mandatory ONLY on the Incident table, but leaving it optional on the generic Task table.